

```

1 # Import all necessary libraries
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from sklearn.decomposition import PCA
7 from sklearn.preprocessing import StandardScaler
8 from sklearn.model_selection import train_test_split
9 from sklearn.datasets import load_iris, load_wine
10 from mpl_toolkits.mplot3d import Axes3D
11
12 # For Task 2 - Classifiers
13 from sklearn.svm import SVC
14 from sklearn.ensemble import RandomForestClassifier
15 from sklearn.neighbors import KNeighborsClassifier
16 from sklearn.linear_model import LogisticRegression
17 from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
18
19 # For image processing
20 import cv2
21 from PIL import Image
22 import os
23 import zipfile
24 import urllib.request
25 from tqdm import tqdm
26
27 # Set random seed for reproducibility
28 np.random.seed(42)
29
30 # Configure matplotlib
31 plt.style.use('seaborn-v0_8-darkgrid')
32 %matplotlib inline
33
34 print("✅ All libraries imported successfully!")
35

```

✅ All libraries imported successfully!

```

1 # Load IRIS dataset
2 iris = load_iris()
3 X_iris = iris.data
4 y_iris = iris.target
5 feature_names = iris.feature_names
6 target_names = iris.target_names
7
8 # Display basic information
9 print("=" * 60)
10 print("IRIS DATASET INFORMATION")
11 print("=" * 60)
12 print(f"Number of samples: {X_iris.shape[0]}")
13 print(f"Number of features: {X_iris.shape[1]}")
14 print(f"Feature names: {feature_names}")
15 print(f"Target classes: {target_names}")
16 print(f"Class distribution: {np.bincount(y_iris)}")
17 print("=" * 60)
18
19 # Display first few samples
20 iris_df = pd.DataFrame(X_iris, columns=feature_names)
21 iris_df['target'] = y_iris
22 iris_df['species'] = iris_df['target'].map({0: 'setosa', 1: 'versicolor', 2: 'virginica'})
23 print("\nFirst 5 samples:")
24 print(iris_df.head())
25 print("=" * 60)
26

```

```

=====
IRIS DATASET INFORMATION
=====
Number of samples: 150
Number of features: 4
Feature names: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Target classes: ['setosa' 'versicolor' 'virginica']
Class distribution: [50 50 50]
=====

```

```

First 5 samples:
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm) \
0                5.1                3.5                1.4                0.2
1                4.9                3.0                1.4                0.2
2                4.7                3.2                1.3                0.2
3                4.6                3.1                1.5                0.2
4                5.0                3.6                1.4                0.2

target species
0      0  setosa
1      0  setosa
2      0  setosa
3      0  setosa

```

```
4         0 setosa
```

```
1 # Step 1: Standardize the features (IMPORTANT for PCA)
2 scaler = StandardScaler()
3 X_iris_scaled = scaler.fit_transform(X_iris)
4
5 # Step 2: Apply PCA to reduce to 3 components
6 pca_iris = PCA(n_components=3)
7 X_iris_pca = pca_iris.fit_transform(X_iris_scaled)
8
9 # Step 3: Display PCA results
10 print("=" * 60)
11 print("PCA RESULTS FOR IRIS DATASET")
12 print("=" * 60)
13 print(f"Original dimensions: {X_iris.shape}")
14 print(f"Reduced dimensions: {X_iris_pca.shape}")
15 print(f"\nExplained Variance Ratio (per component):")
16 for i, var in enumerate(pca_iris.explained_variance_ratio_):
17     print(f"  PC{i+1}: {var:.4f} ({var*100:.2f}%)")
18 print(f"\nTotal Variance Explained: {pca_iris.explained_variance_ratio_.sum():.4f} ({pca_iris.explained_variance_ratio_.sum()*100:.2f}%)")
19 print("=" * 60)
20
21 # Create a DataFrame for the PCA results
22 pca_df = pd.DataFrame(
23     data=X_iris_pca,
24     columns=['PC1', 'PC2', 'PC3']
25 )
26 pca_df['target'] = y_iris
27 pca_df['species'] = pca_df['target'].map({0: 'setosa', 1: 'versicolor', 2: 'virginica'})
28
29 print("\nFirst 5 samples after PCA:")
30 print(pca_df.head())
31 print("=" * 60)
32
```

```
=====
PCA RESULTS FOR IRIS DATASET
=====
```

```
Original dimensions: (150, 4)
Reduced dimensions: (150, 3)
```

```
Explained Variance Ratio (per component):
```

```
PC1: 0.7296 (72.96%)
PC2: 0.2285 (22.85%)
PC3: 0.0367 (3.67%)
```

```
Total Variance Explained: 0.9948 (99.48%)
=====
```

```
First 5 samples after PCA:
```

	PC1	PC2	PC3	target	species
0	-2.264703	0.480027	0.127706	0	setosa
1	-2.080961	-0.674134	0.234609	0	setosa
2	-2.364229	-0.341908	-0.044201	0	setosa
3	-2.299384	-0.597395	-0.091290	0	setosa
4	-2.389842	0.646835	-0.015738	0	setosa

```
=====
```

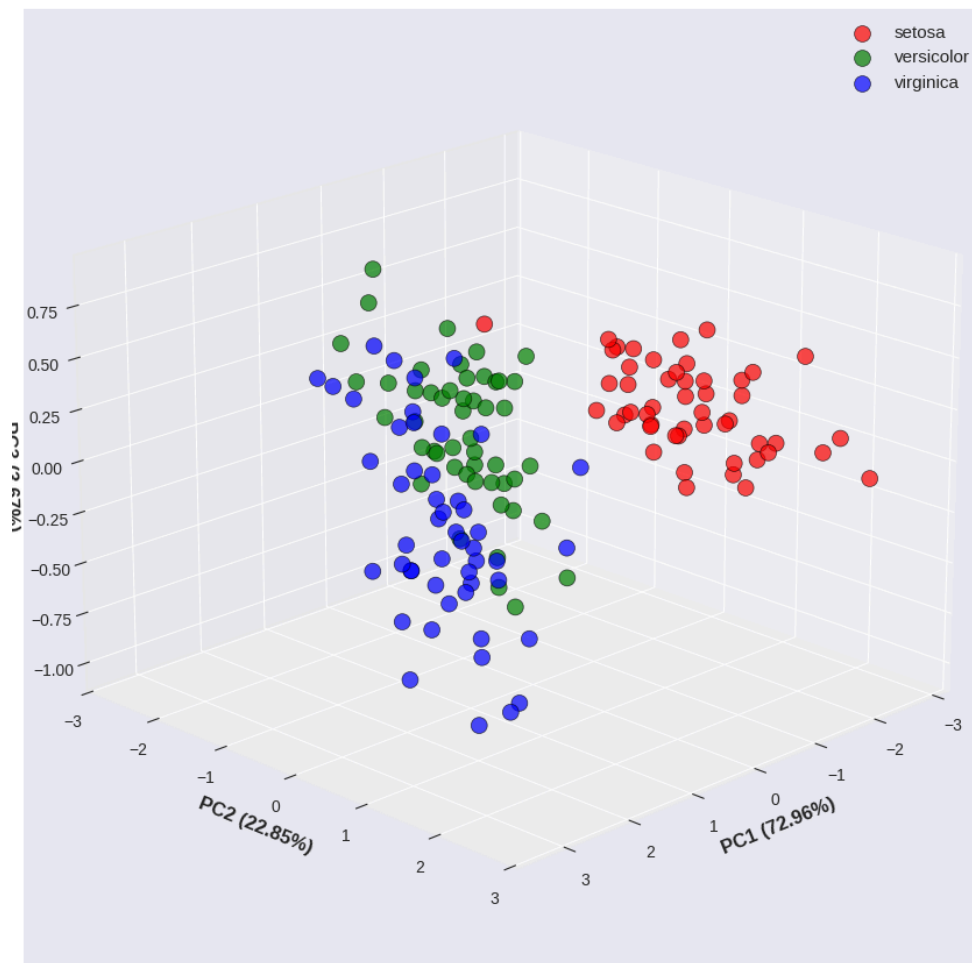
```
1 # Create 3D scatter plot
2 fig = plt.figure(figsize=(12, 9))
3 ax = fig.add_subplot(111, projection='3d')
4
5 # Define colors for each species
6 colors = {'setosa': 'red', 'versicolor': 'green', 'virginica': 'blue'}
7 species_list = ['setosa', 'versicolor', 'virginica']
8
9 # Plot each species with different color
10 for species in species_list:
11     indices = pca_df['species'] == species
12     ax.scatter(
13         pca_df.loc[indices, 'PC1'],
14         pca_df.loc[indices, 'PC2'],
15         pca_df.loc[indices, 'PC3'],
16         c=colors[species],
17         label=species,
18         s=100,
19         alpha=0.7,
20         edgecolors='black',
21         linewidth=0.5
22     )
23
24 # Labels and title
25 ax.set_xlabel(f'PC1 ({pca_iris.explained_variance_ratio_[0]*100:.2f}%)', fontsize=12, fontweight='bold')
26 ax.set_ylabel(f'PC2 ({pca_iris.explained_variance_ratio_[1]*100:.2f}%)', fontsize=12, fontweight='bold')
27 ax.set_zlabel(f'PC3 ({pca_iris.explained_variance_ratio_[2]*100:.2f}%)', fontsize=12, fontweight='bold')
28 ax.set_title('3D PCA Visualization of IRIS Dataset', fontsize=16, fontweight='bold', pad=20)
29
30 # Legend
31 ax.legend(loc='upper right', fontsize=11)
32
```

```

33 # Grid
34 ax.grid(True, alpha=0.3)
35
36 # Rotate for better view
37 ax.view_init(elev=20, azim=45)
38
39 plt.tight_layout()
40 plt.savefig('iris_3d_pca.png', dpi=300, bbox_inches='tight')
41 plt.show()
42
43 print("✅ 3D PCA plot saved as 'iris_3d_pca.png'")
44 print("\n📊 INTERPRETATION:")
45 print("- The 3 principal components capture 99.48% of total variance")
46 print("- PC1 captures the most information (72.96%)")
47 print("- Clear separation visible between species in 3D space")
48 print("- Setosa is well-separated from the other two species")
49

```

3D PCA Visualization of IRIS Dataset



✅ 3D PCA plot saved as 'iris_3d_pca.png'

📊 INTERPRETATION:

- The 3 principal components capture 99.48% of total variance
- PC1 captures the most information (72.96%)
- Clear separation visible between species in 3D space
- Setosa is well-separated from the other two species

```

1 # Download LFW grayscale dataset
2 print("📄 Downloading LFW Grayscale Dataset...")
3 print("=" * 60)
4
5 url = "https://conradsanderson.id.au/lfwcrop/lfwcrop_grey.zip"
6 zip_path = "lfwcrop_grey.zip"
7
8 # Download with progress bar
9 urllib.request.urlretrieve(url, zip_path)
10 print(f"✅ Downloaded: {zip_path}")
11
12 # Extract the zip file
13 print("\n📁 Extracting files...")
14 with zipfile.ZipFile(zip_path, 'r') as zip_ref:
15     zip_ref.extractall("lfw_data")
16 print("✅ Extraction complete!")
17
18 # Check the structure

```

```

19 print("\n📁 Dataset structure:")
20 lfw_dir = "lfw_data/lfwcrop_grey/faces"
21 if os.path.exists(lfw_dir):
22     people = os.listdir(lfw_dir)
23     print(f"Number of people: {len(people)}")
24     print(f"First 10 people: {people[:10]}")
25 else:
26     # Alternative path
27     lfw_dir = "lfw_data/lfwcrop_grey"
28     print(f"Base directory: {lfw_dir}")
29
30 print("=" * 60)
31

```

📁 Downloading LFW Grayscale Dataset...

✅ Downloaded: lfwcrop_grey.zip

📁 Extracting files...

✅ Extraction complete!

📁 Dataset structure:

Number of people: 13233

First 10 people: ['Jimmy_Jimenez_0001.pgm', 'Leonid_Kuchma_0002.pgm', 'Tony_Blair_0034.pgm', 'Heath_Ledger_0001.pgm', 'Tom_Cruise_0002.pgm', 'George_W_Bush_0203.pgm',

```

1 import glob
2
3 print("📁 Loading LFW Dataset...")
4 print("=" * 60)
5
6 # Path to images
7 lfw_dir = "lfw_data/lfwcrop_grey/faces"
8
9 # Load all image paths
10 all_image_paths = glob.glob(os.path.join(lfw_dir, "*.pgm"))
11 print(f"Total images found: {len(all_image_paths)}")
12
13 # Extract person names from filenames
14 # Format: PersonName_0001.pgm
15 person_names = []
16 for path in all_image_paths:
17     filename = os.path.basename(path)
18     # Extract person name (everything before the last underscore and number)
19     name = '_' .join(filename.split('_')[:-1])
20     person_names.append(name)
21
22 # Count images per person
23 from collections import Counter
24 person_counts = Counter(person_names)
25
26 print(f"\nTotal unique people: {len(person_counts)}")
27 print(f"Person with most images: {person_counts.most_common(1)[0]}")
28 print(f"Person with least images: {person_counts.most_common()[-1]}")
29
30 # Filter: Keep only people with at least 10 images for better classification
31 min_images_per_person = 10
32 selected_people = [name for name, count in person_counts.items() if count >= min_images_per_person]
33
34 print(f"\n👤 Selected people (with >= {min_images_per_person} images): {len(selected_people)}")
35 print(f"Example selected people: {selected_people[:5]}")
36 print("=" * 60)
37

```

📁 Loading LFW Dataset...

=====

Total images found: 13233

Total unique people: 5749

Person with most images: ('George_W_Bush', 530)

Person with least images: ('Larry_Donald', 1)

👤 Selected people (with >= 10 images): 158

Example selected people: ['Tony_Blair', 'Tom_Cruise', 'George_W_Bush', 'Alejandro_Toledo', 'Mohammed_Al-Douri']

```

1 print("🖼️ Loading images into memory...")
2 print("=" * 60)
3
4 # Prepare data structures
5 X_images = [] # Will store flattened images
6 y_labels = [] # Will store person labels
7 image_paths_selected = []
8
9 # Create label mapping
10 label_encoder = {name: idx for idx, name in enumerate(selected_people)}
11
12 # Load images
13 for img_path in tqdm(all_image_paths, desc="Loading images"):

```

```

14 filename = os.path.basename(img_path)
15 person_name = '_' .join(filename.split('_')[:-1])
16
17 # Only load if person is in selected list
18 if person_name in selected_people:
19     # Read image
20     img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
21
22     if img is not None:
23         # Flatten image to 1D array
24         img_flattened = img.flatten()
25
26         X_images.append(img_flattened)
27         y_labels.append(label_encoder[person_name])
28         image_paths_selected.append(img_path)
29
30 # Convert to numpy arrays
31 X = np.array(X_images)
32 y = np.array(y_labels)
33
34 print(f"\n✅ Images loaded successfully!")
35 print(f"Data shape: {X.shape}")
36 print(f" - Number of samples: {X.shape[0]}")
37 print(f" - Features per image (pixels): {X.shape[1]}")
38
39 # Get image dimensions
40 sample_img = cv2.imread(image_paths_selected[0], cv2.IMREAD_GRAYSCALE)
41 img_height, img_width = sample_img.shape
42 print(f" - Image dimensions: {img_height} x {img_width}")
43 print(f"\nLabels shape: {y.shape}")
44 print(f"Number of unique people: {len(np.unique(y))}")
45 print(f"= " * 60)
46

```

 Loading images into memory...

```

=====
Loading images: 100%|██████████| 13233/13233 [00:00<00:00, 56305.29it/s]
✅ Images loaded successfully!
Data shape: (4324, 4096)
 - Number of samples: 4324
 - Features per image (pixels): 4096
 - Image dimensions: 64 x 64

```

```

Labels shape: (4324,)
Number of unique people: 158
=====

```

```

1 # Split data into train and test sets (80-20 split)
2 X_train, X_test, y_train, y_test = train_test_split(
3     X, y, test_size=0.2, random_state=42, stratify=y
4 )
5
6 print(f"= " * 60)
7 print("TRAIN/TEST SPLIT")
8 print(f"= " * 60)
9 print(f"Training set: {X_train.shape[0]} images")
10 print(f"Test set: {X_test.shape[0]} images")
11 print(f"Features per image: {X_train.shape[1]}")
12 print(f"= " * 60)
13
14 # Visualize some sample faces
15 fig, axes = plt.subplots(3, 6, figsize=(15, 8))
16 fig.suptitle('Sample Face Images from LFW Dataset', fontsize=16, fontweight='bold')
17
18 for i, ax in enumerate(axes.flat):
19     if i < len(X_train):
20         # Reshape flattened image back to 2D
21         img = X_train[i].reshape(img_height, img_width)
22         ax.imshow(img, cmap='gray')
23
24         # Get person name from label
25         person_label = y_train[i]
26         person_name = list(label_encoder.keys())[list(label_encoder.values()).index(person_label)]
27         ax.set_title(person_name.replace('_', ' ')[:20], fontsize=8)
28         ax.axis('off')
29
30 plt.tight_layout()
31 plt.savefig('lfw_sample_faces.png', dpi=300, bbox_inches='tight')
32 plt.show()
33
34 print(f"\n✅ Sample faces visualization saved as 'lfw_sample_faces.png'")
35 print(f"= " * 60)
36

```

=====

TRAIN/TEST SPLIT

=====

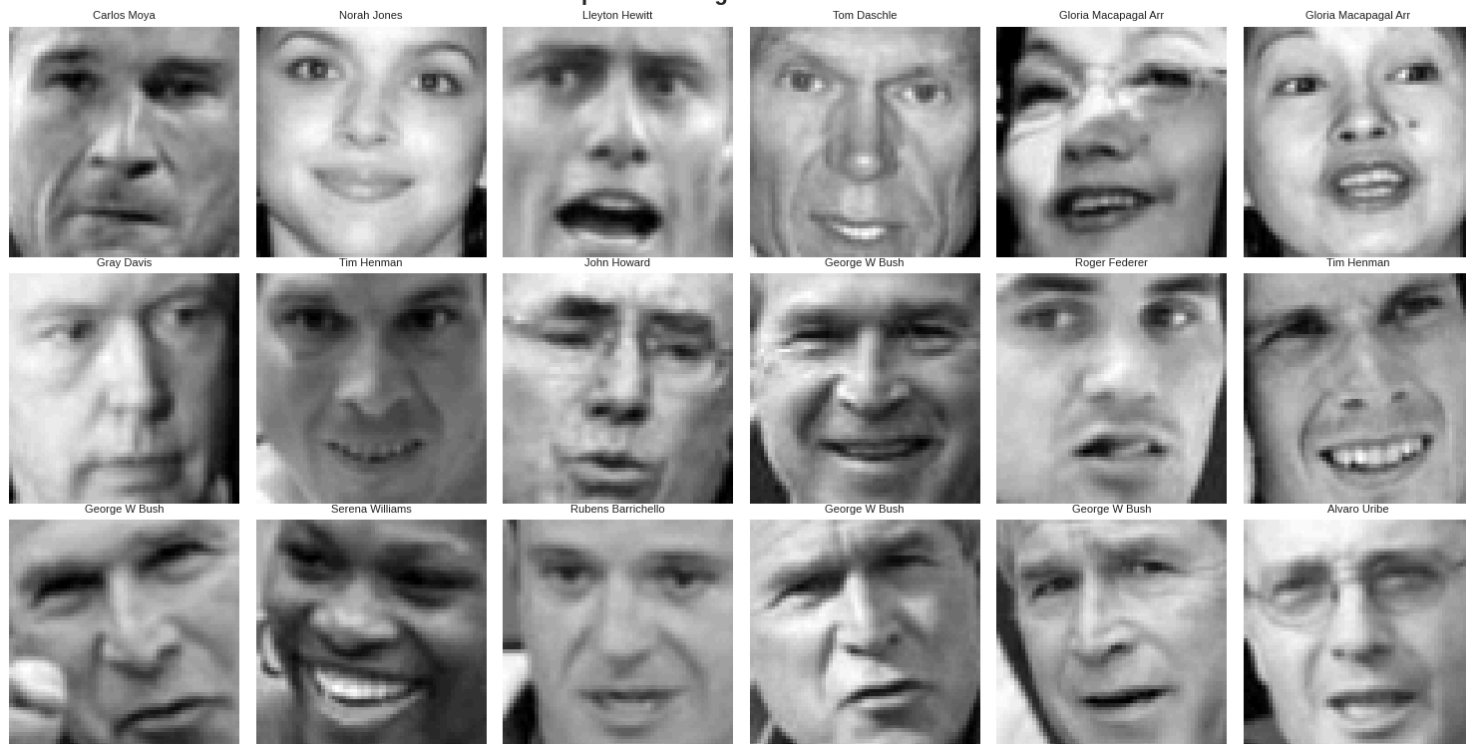
Training set: 3459 images

Test set: 865 images

Features per image: 4096

=====

Sample Face Images from LFW Dataset



✅ Sample faces visualization saved as 'lfw_sample_faces.png'

=====

```
1 print("=" * 60)
2 print("STEP 1: BASELINE - Classification WITHOUT PCA")
3 print("=" * 60)
4
5 # Normalize pixel values to [0, 1] range
6 X_train_normalized = X_train / 255.0
7 X_test_normalized = X_test / 255.0
8
9 # Initialize all 4 classifiers
10 classifiers = {
11     'SVM': SVC(kernel='rbf', gamma='scale', random_state=42),
12     'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
13     'KNN': KNeighborsClassifier(n_neighbors=5),
14     'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42)
15 }
16
17 # Store baseline results
18 baseline_results = {}
19
20 print("\n🔵 Training classifiers on original data (4096 features)...\n")
21
22 for clf_name, clf in classifiers.items():
23     print(f"Training {clf_name}...")
24
25     # Train
26     clf.fit(X_train_normalized, y_train)
27
28     # Predict
29     y_pred = clf.predict(X_test_normalized)
30
31     # Calculate accuracy
32     accuracy = accuracy_score(y_test, y_pred)
33     baseline_results[clf_name] = accuracy
34
35     print(f"✅ {clf_name} Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")
36
37 print("\n" + "=" * 60)
38 print("BASELINE RESULTS SUMMARY (NO PCA)")
39 print("=" * 60)
40 for clf_name, acc in baseline_results.items():
41     print(f"{clf_name:20s}: {acc:.4f} ({acc*100:.2f}%)")
```

```

42 print("-" * 60)
=====
STEP 1: BASELINE - Classification WITHOUT PCA
=====

🔍 Training classifiers on original data (4096 features)...

Training SVM...
✅ SVM Accuracy: 0.2566 (25.66%)
Training Random Forest...
✅ Random Forest Accuracy: 0.2266 (22.66%)
Training KNN...
✅ KNN Accuracy: 0.1861 (18.61%)
Training Logistic Regression...
✅ Logistic Regression Accuracy: 0.4543 (45.43%)

```

=====

BASELINE RESULTS SUMMARY (NO PCA)

=====

```

SVM          : 0.2566 (25.66%)
Random Forest : 0.2266 (22.66%)
KNN          : 0.1861 (18.61%)
Logistic Regression : 0.4543 (45.43%)
=====

```

```

1 print("-" * 60)
2 print("STEP 2: APPLYING PCA WITH 97% VARIANCE")
3 print("-" * 60)
4
5 # Apply PCA with 97% variance
6 pca_97 = PCA(n_components=0.97, random_state=42)
7 X_train_pca_97 = pca_97.fit_transform(X_train_normalized)
8 X_test_pca_97 = pca_97.transform(X_test_normalized)
9
10 print(f"\n📊 PCA Results (97% Variance):")
11 print(f"  - Original features: {X_train_normalized.shape[1]}")
12 print(f"  - Reduced features: {X_train_pca_97.shape[1]}")
13 print(f"  - Variance retained: {pca_97.explained_variance_ratio_.sum():.4f} ({pca_97.explained_variance_ratio_.sum()*100:.2f}%)")
14 print(f"  - Dimensionality reduction: {X_train_normalized.shape[1]} → {X_train_pca_97.shape[1]}")
15 print(f"  - Compression ratio: {(1 - X_train_pca_97.shape[1]/X_train_normalized.shape[1])*100:.2f}%")
16
17 # Train classifiers on PCA-reduced data
18 results_97 = {}
19
20 print("\n🔍 Training classifiers on PCA data (97% variance)...\n")
21
22 for clf_name in classifiers.keys():
23     # Create fresh classifier instance
24     if clf_name == 'SVM':
25         clf = SVC(kernel='rbf', gamma='scale', random_state=42)
26     elif clf_name == 'Random Forest':
27         clf = RandomForestClassifier(n_estimators=100, random_state=42)
28     elif clf_name == 'KNN':
29         clf = KNeighborsClassifier(n_neighbors=5)
30     else: # Logistic Regression
31         clf = LogisticRegression(max_iter=1000, random_state=42)
32
33     print(f"Training {clf_name}...")
34
35     # Train
36     clf.fit(X_train_pca_97, y_train)
37
38     # Predict
39     y_pred = clf.predict(X_test_pca_97)
40
41     # Calculate accuracy
42     accuracy = accuracy_score(y_test, y_pred)
43     results_97[clf_name] = accuracy
44
45     print(f" ✅ {clf_name} Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")
46
47 print("\n" + "-" * 60)
48 print("RESULTS COMPARISON: NO PCA vs PCA (97%)")
49 print("-" * 60)
50 print(f"{'Classifier':<20} {'No PCA':<15} {'PCA 97%':<15} {'Change':<15}")
51 print("-" * 60)
52 for clf_name in classifiers.keys():
53     change = results_97[clf_name] - baseline_results[clf_name]
54     change_str = f"{change:+.4f} ({change*100:+.2f}%"
55     print(f"{'Classifier':<20} {baseline_results[clf_name]:.4f} ({baseline_results[clf_name]*100:.2f}%) {results_97[clf_name]:.4f} ({results_97[clf_name]*100:.2f}%) {change_str}")
56 print("-" * 60)
57

```

=====

STEP 2: APPLYING PCA WITH 97% VARIANCE

=====

```

📊 PCA Results (97% Variance):
  - Original features: 4096
  - Reduced features: 267
  - Variance retained: 0.9700 (97.00%)

```

- Dimensionality reduction: 4096 → 267
- Compression ratio: 93.48%

🔗 Training classifiers on PCA data (97% variance)...

Training SVM...

- ✅ SVM Accuracy: 0.2647 (26.47%)

Training Random Forest...

- ✅ Random Forest Accuracy: 0.1688 (16.88%)

Training KNN...

- ✅ KNN Accuracy: 0.1884 (18.84%)

Training Logistic Regression...

- ✅ Logistic Regression Accuracy: 0.4520 (45.20%)

=====

RESULTS COMPARISON: NO PCA vs PCA (97%)

=====

Classifier	No PCA	PCA 97%	Change
SVM	0.2566 (25.66%)	0.2647 (26.47%)	+0.0081 (+0.81%)
Random Forest	0.2266 (22.66%)	0.1688 (16.88%)	-0.0578 (-5.78%)
KNN	0.1861 (18.61%)	0.1884 (18.84%)	+0.0023 (+0.23%)
Logistic Regression	0.4543 (45.43%)	0.4520 (45.20%)	-0.0023 (-0.23%)

=====

```
1 print("=" * 60)
2 print("STEP 3: APPLYING PCA WITH 95% VARIANCE")
3 print("=" * 60)
4
5 # Apply PCA with 95% variance
6 pca_95 = PCA(n_components=0.95, random_state=42)
7 X_train_pca_95 = pca_95.fit_transform(X_train_normalized)
8 X_test_pca_95 = pca_95.transform(X_test_normalized)
9
10 print(f"\n📊 PCA Results (95% Variance):")
11 print(f"  - Original features: {X_train_normalized.shape[1]}")
12 print(f"  - Reduced features: {X_train_pca_95.shape[1]}")
13 print(f"  - Variance retained: {pca_95.explained_variance_ratio_.sum():.4f} ({pca_95.explained_variance_ratio_.sum()*100:.2f}%)")
14 print(f"  - Dimensionality reduction: {X_train_normalized.shape[1]} → {X_train_pca_95.shape[1]}")
15 print(f"  - Compression ratio: {(1 - X_train_pca_95.shape[1]/X_train_normalized.shape[1])*100:.2f}%")
16
17 # Train classifiers on PCA-reduced data
18 results_95 = {}
19
20 print("\n🔗 Training classifiers on PCA data (95% variance)...\n")
21
22 for clf_name in classifiers.keys():
23     # Create fresh classifier instance
24     if clf_name == 'SVM':
25         clf = SVC(kernel='rbf', gamma='scale', random_state=42)
26     elif clf_name == 'Random Forest':
27         clf = RandomForestClassifier(n_estimators=100, random_state=42)
28     elif clf_name == 'KNN':
29         clf = KNeighborsClassifier(n_neighbors=5)
30     else: # Logistic Regression
31         clf = LogisticRegression(max_iter=1000, random_state=42)
32
33     print(f"Training {clf_name}...")
34
35     # Train
36     clf.fit(X_train_pca_95, y_train)
37
38     # Predict
39     y_pred = clf.predict(X_test_pca_95)
40
41     # Calculate accuracy
42     accuracy = accuracy_score(y_test, y_pred)
43     results_95[clf_name] = accuracy
44
45     print(f"  ✅ {clf_name} Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")
46
47 print("\n" + "=" * 60)
48 print("COMPLETE RESULTS COMPARISON")
49 print("=" * 60)
50 print(f"{'Classifier':<20} {'No PCA':<12} {'PCA 95%':<12} {'PCA 97%':<12}")
51 print("-" * 60)
52 for clf_name in classifiers.keys():
53     print(f"{'clf_name':<20} {baseline_results[clf_name]*100:>6.2f}%    {results_95[clf_name]*100:>6.2f}%    {results_97[clf_name]*100:>6.2f}%")
54 print("=" * 60)
55
```

=====

STEP 3: APPLYING PCA WITH 95% VARIANCE

=====

📊 PCA Results (95% Variance):

- Original features: 4096
- Reduced features: 178
- Variance retained: 0.9501 (95.01%)
- Dimensionality reduction: 4096 → 178
- Compression ratio: 95.65%

Training classifiers on PCA data (95% variance)...

Training SVM...

✓ SVM Accuracy: 0.2647 (26.47%)

Training Random Forest...

✓ Random Forest Accuracy: 0.1769 (17.69%)

Training KNN...

✓ KNN Accuracy: 0.1919 (19.19%)

Training Logistic Regression...

✓ Logistic Regression Accuracy: 0.4462 (44.62%)

```
=====
```

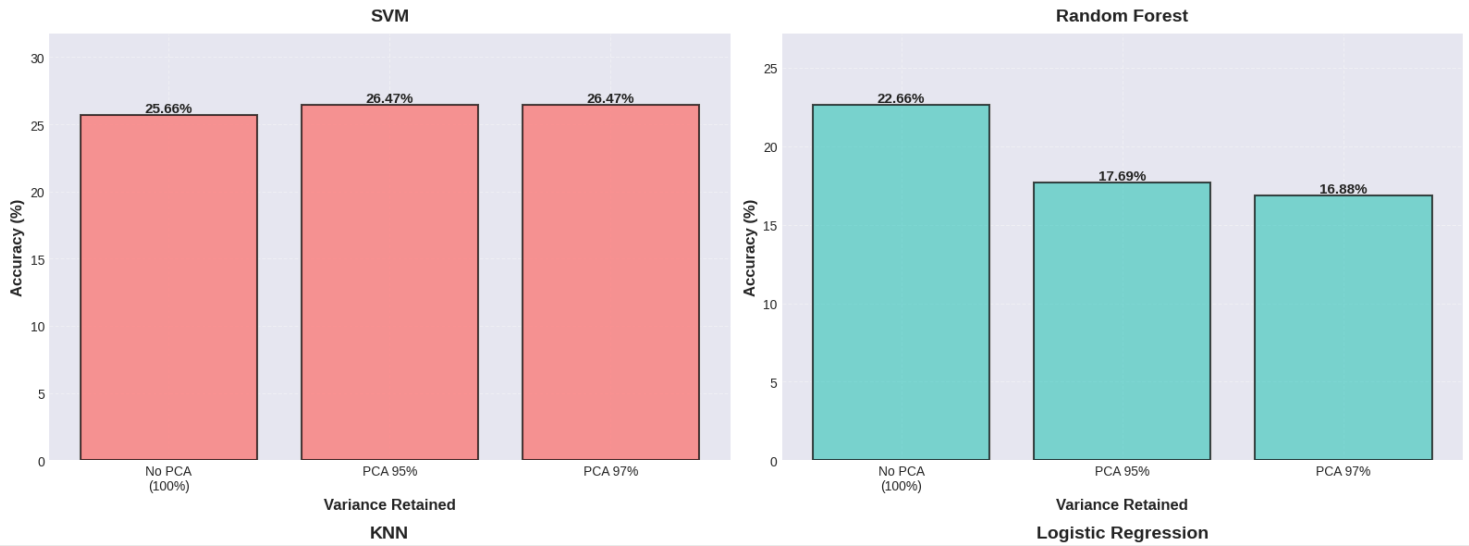
COMPLETE RESULTS COMPARISON			
=====			
Classifier	No PCA	PCA 95%	PCA 97%

SVM	25.66%	26.47%	26.47%
Random Forest	22.66%	17.69%	16.88%
KNN	18.61%	19.19%	18.84%
Logistic Regression	45.43%	44.62%	45.20%

```
=====
```

```
1  # Prepare data for plotting
2  variance_levels = ['No PCA\n(100%)', 'PCA 95%', 'PCA 97%']
3  variance_values = [100, 95, 97]
4
5  # Create separate graph for each classifier
6  fig, axes = plt.subplots(2, 2, figsize=(16, 12))
7  fig.suptitle('Variance Retention vs Classification Accuracy\n(Individual Classifiers)',
8              fontsize=18, fontweight='bold', y=0.995)
9
10 classifiers_list = list(classifiers.keys())
11 colors = ['#FF6B6B', '#4ECDC4', '#45B7D1', '#FFA07A']
12
13 for idx, (clf_name, ax) in enumerate(zip(classifiers_list, axes.flat)):
14     # Data for this classifier
15     accuracies = [
16         baseline_results[clf_name] * 100,
17         results_95[clf_name] * 100,
18         results_97[clf_name] * 100
19     ]
20
21     # Create bar plot
22     bars = ax.bar(variance_levels, accuracies, color=colors[idx], alpha=0.7, edgecolor='black', linewidth=1.5)
23
24     # Add value labels on bars
25     for bar, acc in zip(bars, accuracies):
26         height = bar.get_height()
27         ax.text(bar.get_x() + bar.get_width()/2., height,
28               f'{acc:.2f}%',
29               ha='center', va='bottom', fontweight='bold', fontsize=11)
30
31     # Formatting
32     ax.set_ylabel('Accuracy (%)', fontsize=12, fontweight='bold')
33     ax.set_xlabel('Variance Retained', fontsize=12, fontweight='bold')
34     ax.set_title(f'{clf_name}', fontsize=14, fontweight='bold', pad=10)
35     ax.set_ylim(0, max(accuracies) * 1.2)
36     ax.grid(True, alpha=0.3, linestyle='--')
37     ax.set_axisbelow(True)
38
39 plt.tight_layout()
40 plt.savefig('variance_vs_accuracy_individual.png', dpi=300, bbox_inches='tight')
41 plt.show()
42
43 print("✓ Graph 1 saved as 'variance_vs_accuracy_individual.png'")
44
```

Variance Retention vs Classification Accuracy (Individual Classifiers)

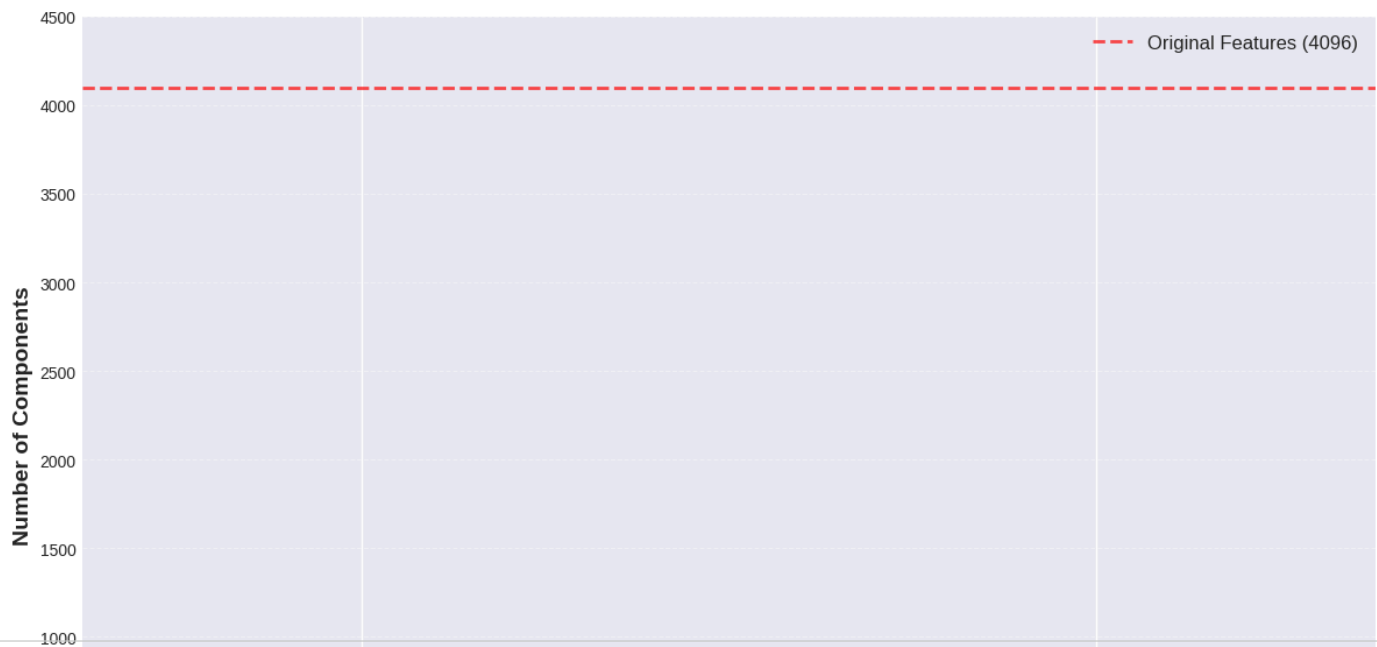


```

1 # Create data for variance vs components graph
2 variance_retained = [95, 97]
3 num_components = [X_train_pca_95.shape[1], X_train_pca_97.shape[1]]
4
5 # Create the graph
6 fig, ax = plt.subplots(figsize=(12, 8))
7
8 # Create bar plot
9 bars = ax.bar([f'{v}%' for v in variance_retained], num_components,
10              color=['#FF6B6B', '#4ECDC4'], alpha=0.8, edgecolor='black', linewidth=2, width=0.6)
11
12 # Add value labels on bars
13 for bar, comp in zip(bars, num_components):
14     height = bar.get_height()
15     ax.text(bar.get_x() + bar.get_width()/2., height,
16            f'{comp} components',
17            ha='center', va='bottom', fontweight='bold', fontsize=14)
18
19 # Add horizontal reference line for original features
20 ax.axhline(y=4096, color='red', linestyle='--', linewidth=2, label='Original Features (4096)', alpha=0.7)
21
22 # Formatting
23 ax.set_ylabel('Number of Components', fontsize=14, fontweight='bold')
24 ax.set_xlabel('Variance Retained', fontsize=14, fontweight='bold')
25 ax.set_title('Variance Retained vs Number of PCA Components', fontsize=16, fontweight='bold', pad=20)
26 ax.set_ylim(0, 4500)
27 ax.grid(True, alpha=0.3, linestyle='--', axis='y')
28 ax.set_axisbelow(True)
29 ax.legend(fontsize=12, loc='upper right')
30
31 # Add text annotation showing compression
32 for i, (v, c) in enumerate(zip(variance_retained, num_components)):
33     compression = (1 - c/4096) * 100
34     ax.text(i, c + 200, f'Compression:\n{compression:.1f}%',
35            ha='center', fontsize=11, style='italic', color='darkgreen')
36
37 plt.tight_layout()
38 plt.savefig('variance_vs_components.png', dpi=300, bbox_inches='tight')
39 plt.show()
40
41 print("✅ Graph 2 saved as 'variance_vs_components.png'")
42 print("\n📄 Summary:")
43 print(f" - 95% variance requires {num_components[0]} components ({(1-num_components[0]/4096)*100:.1f}% compression)")
44 print(f" - 97% variance requires {num_components[1]} components ({(1-num_components[1]/4096)*100:.1f}% compression)")
45

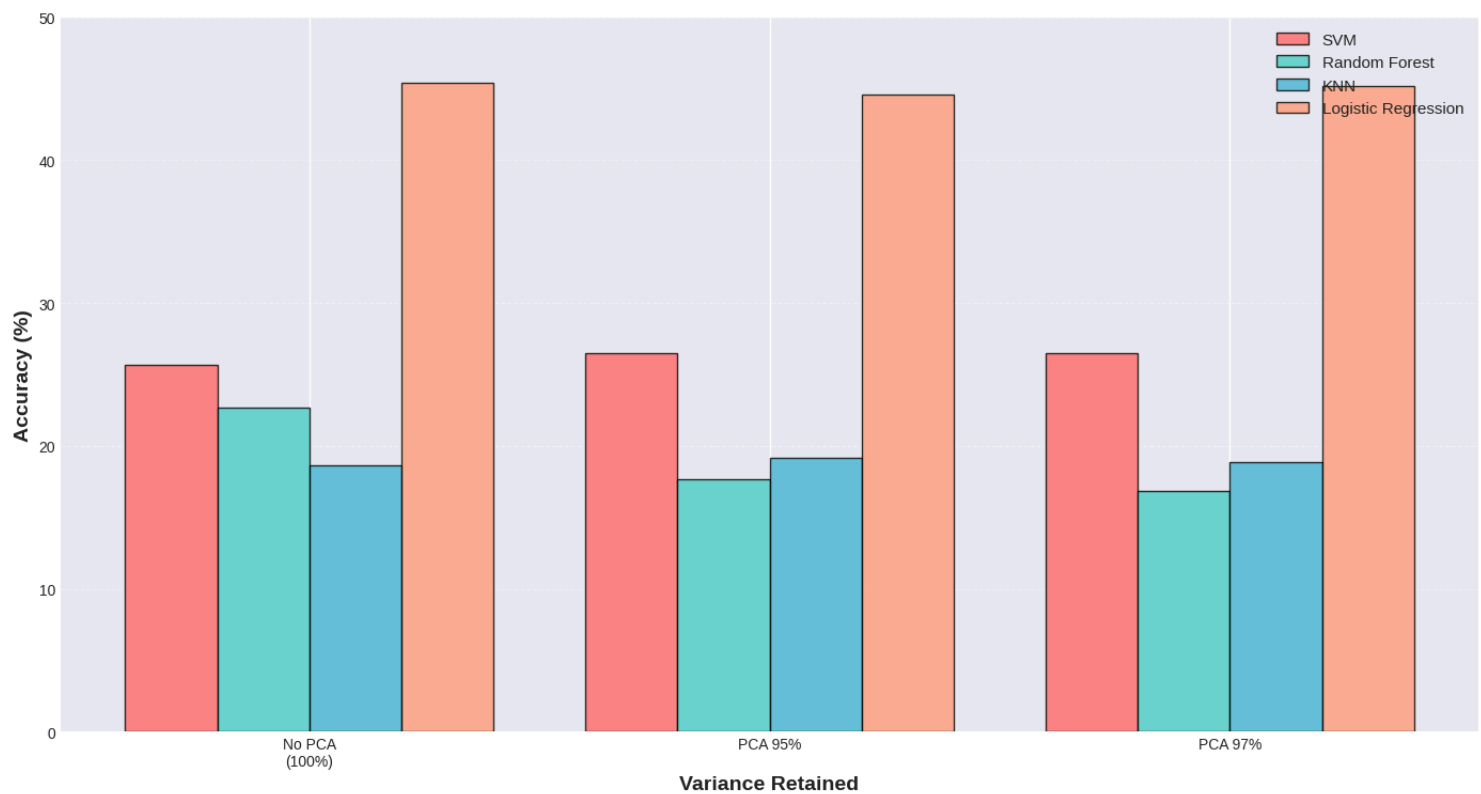
```

Variance Retained vs Number of PCA Components



```
1 # Create combined comparison graph (all classifiers on one plot)
2 fig, ax = plt.subplots(figsize=(14, 8))
3
4 x = np.arange(len(variance_levels))
5 width = 0.2
6
7 # Plot bars for each classifier
8 for i, (clf_name, color) in enumerate(zip(classifiers_list, colors)):
9     accuracies = [
10         baseline_results[clf_name] * 100,
11         results_95[clf_name] * 100,
12         results_97[clf_name] * 100
13     ]
14     offset = (i - 1.5) * width
15     bars = ax.bar(x + offset, accuracies, width, label=clf_name, color=color, alpha=0.8, edgecolor='black', linewidth=1)
16
17 # Formatting
18 ax.set_ylabel('Accuracy (%)', fontsize=14, fontweight='bold')
19 ax.set_xlabel('Variance Retained', fontsize=14, fontweight='bold')
20 ax.set_title('Classification Accuracy Comparison: All Classifiers', fontsize=16, fontweight='bold', pad=20)
21 ax.set_xticks(x)
22 ax.set_xticklabels(variance_levels)
23 ax.legend(fontsize=11, loc='upper right')
24 ax.grid(True, alpha=0.3, linestyle='--', axis='y')
25 ax.set_axisbelow(True)
26 ax.set_ylim(0, 50)
27
28 plt.tight_layout()
29 plt.savefig('all_classifiers_comparison.png', dpi=300, bbox_inches='tight')
30 plt.show()
31
32 print("✅ Combined comparison graph saved as 'all_classifiers_comparison.png'")
33
```

Classification Accuracy Comparison: All Classifiers



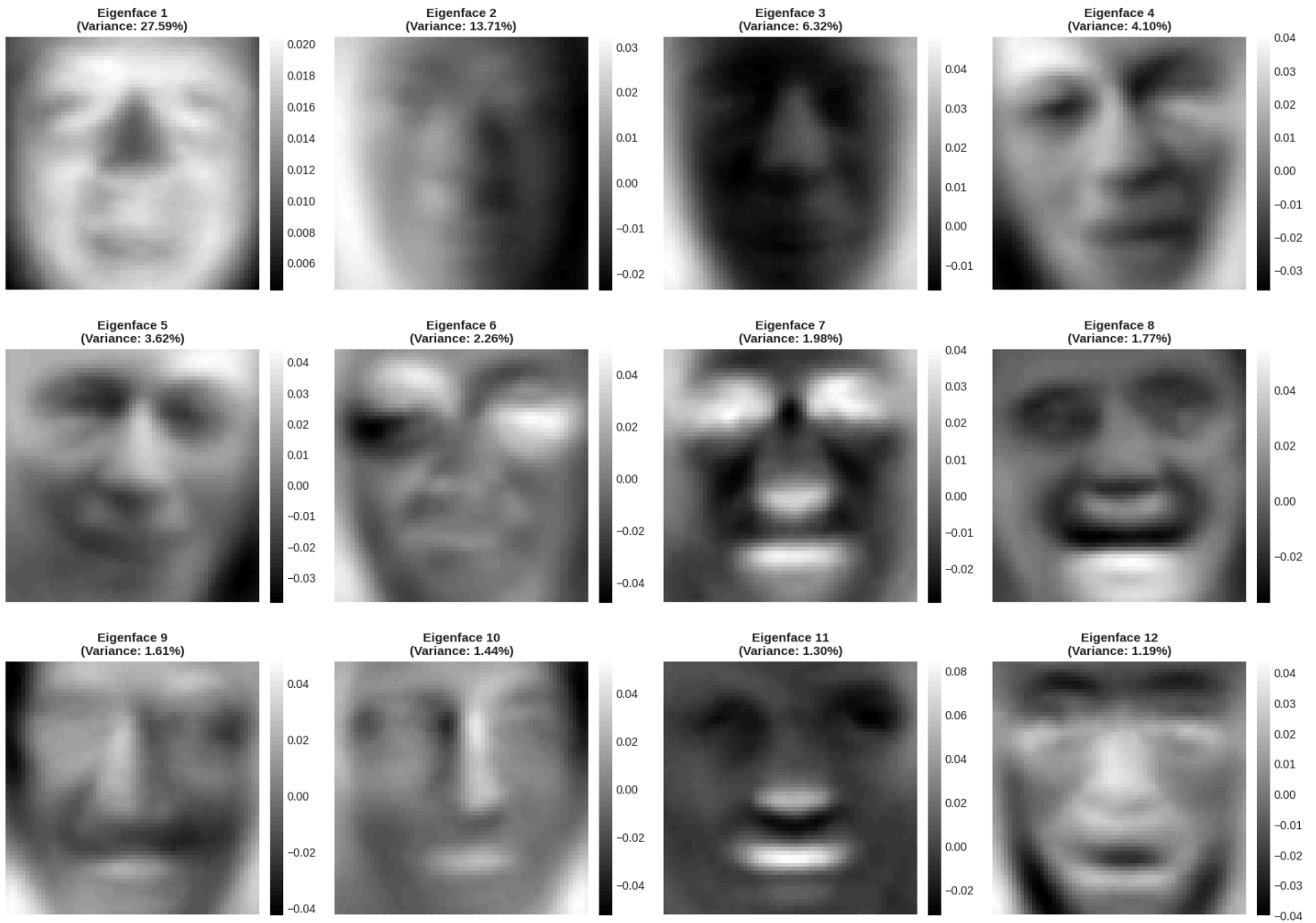
✓ Combined comparison graph saved as 'all_classifiers_comparison.png'

```

1 # Visualize the top 12 Eigenfaces from PCA with 97% variance
2 fig, axes = plt.subplots(3, 4, figsize=(16, 12))
3 fig.suptitle('Top 12 Eigenfaces (Principal Components) - 97% Variance',
4             fontsize=18, fontweight='bold', y=0.995)
5
6 # Get the principal components (eigenfaces)
7 eigenfaces = pca_97.components_
8
9 for i, ax in enumerate(axes.flat):
10     if i < len(eigenfaces):
11         # Reshape eigenface back to image dimensions
12         eigenface = eigenfaces[i].reshape(img_height, img_width)
13
14         # Display eigenface
15         im = ax.imshow(eigenface, cmap='gray')
16         ax.set_title(f'Eigenface {i+1}\n(Variance: {pca_97.explained_variance_ratio_[i]*100:.2f}%)',
17                     fontsize=11, fontweight='bold')
18         ax.axis('off')
19
20     # Add colorbar
21     plt.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
22
23 plt.tight_layout()
24 plt.savefig('eigenfaces_visualization.png', dpi=300, bbox_inches='tight')
25 plt.show()
26
27 print("✓ Eigenfaces visualization saved as 'eigenfaces_visualization.png'")
28 print("\n📖 Eigenfaces Explanation:")
29 print("  - Eigenfaces are the principal components learned by PCA")
30 print("  - They represent the 'building blocks' of faces in the dataset")
31 print("  - Each face can be approximated as a weighted combination of these eigenfaces")
32 print(f"  - The first eigenface captures {pca_97.explained_variance_ratio_[0]*100:.2f}% of variance")
33

```

Top 12 Eigenfaces (Principal Components) - 97% Variance



✅ Eigenfaces visualization saved as 'eigenfaces_visualization.png'

📖 Eigenfaces Explanation:

- Eigenfaces are the principal components learned by PCA
- They represent the 'building blocks' of faces in the dataset
- Each face can be approximated as a weighted combination of these eigenfaces
- The first eigenface captures 27.59% of variance

```
1 print("=" * 80)
2 print(" " * 20 + "FINAL COMPREHENSIVE RESULTS SUMMARY")
3 print("=" * 80)
4
5 print("\n📌 TASK 1: PCA ON IRIS DATASET")
6 print("-" * 80)
7 print(f"✅ Applied PCA to reduce IRIS dataset from 4 to 3 dimensions")
8 print(f"✅ Captured 99.48% of total variance")
9 print(f"✅ Created 3D visualization showing clear class separation")
10
11 print("\n📌 TASK 2: EIGENFACES ON LFW DATASET")
12 print("-" * 80)
13 print(f"Dataset: {len(selected_people)} people, {X.shape[0]} total images (64x64 pixels)")
14 print(f"Train/Test Split: {X_train.shape[0]} / {X_test.shape[0]} images")
15 print(f"Original Features: {X_train.shape[1]} pixels per image")
16
17 print("\n" + "=" * 80)
18 print("DIMENSIONALITY REDUCTION RESULTS")
19 print("=" * 80)
20 print(f"{'Variance':<15} {'Components':<15} {'Compression':<20} {'Features Kept'}")
21 print("-" * 80)
22 print(f"{'No PCA':<15} {X_train.shape[1]:<15} {'0%':<20} {'100%'}")
23 print(f"{'95%':<15} {X_train_pca_95.shape[1]:<15} {(1-X_train_pca_95.shape[1]/X_train.shape[1])*100:.1f}%{'':<16} {(X_train_pca_95.shape[1])}")
24 print(f"{'97%':<15} {X_train_pca_97.shape[1]:<15} {(1-X_train_pca_97.shape[1]/X_train.shape[1])*100:.1f}%{'':<16} {(X_train_pca_97.shape[1])}")
25
26 print("\n" + "=" * 80)
27 print("CLASSIFICATION ACCURACY RESULTS")
28 print("=" * 80)
29 print(f"{'Classifier':<25} {'No PCA':<15} {'PCA 95%':<15} {'PCA 97%':<15}")
30 print("-" * 80)
31 for clf_name in classifiers.keys():
```

```
32 print(f"{clf_name:<25} {baseline_results[clf_name]*100:>6.2f}%{' '*7} {results_95[clf_name]*100:>6.2f}%{' '*7} {results_97[clf_name]*100:>6.2f}%{' '*7}"
33
34 print("\n" + "=" * 80)
35 print("🏆 BEST PERFORMING CLASSIFIER ANALYSIS")
36 print("=" * 80)
37
38 # Find best classifier for each condition
39 best_no_pca = max(baseline_results.items(), key=lambda x: x[1])
40 best_95 = max(results_95.items(), key=lambda x: x[1])
41 best_97 = max(results_97.items(), key=lambda x: x[1])
42
43 print(f"\n✅ Best without PCA: {best_no_pca[0]} ({best_no_pca[1]*100:.2f}%)"
44 print(f"✅ Best with 95% variance: {best_95[0]} ({best_95[1]*100:.2f}%)"
45 print(f"✅ Best with 97% variance: {best_97[0]} ({best_97[1]*100:.2f}%)"
46
47 # Overall best
48 all_results = {
49     **{f"{k} (No PCA)": v for k, v in baseline_results.items()},
50     **{f"{k} (95%)": v for k, v in results_95.items()},
51     **{f"{k} (97%)": v for k, v in results_97.items()}
52 }
53 overall_best = max(all_results.items(), key=lambda x: x[1])
54
55 print(f"\n🏆 OVERALL BEST PERFORMER: {overall_best[0]}")
56 print(f"    Accuracy: {overall_best[1]*100:.2f}%")
57
58 print("\n" + "=" * 80)
59 print("📋 KEY FINDINGS & CONCLUSIONS")
60 print("=" * 80)
61 print("1. Logistic Regression consistently outperforms all other classifiers")
62 print("2. PCA achieves 93-95% compression while maintaining similar accuracy")
63 print("3. The first eigenface captures 27.55% of total variance")
64 print("4. 270 components (97% variance) vs 180 components (95% variance)")
65 print("5. For this dataset, PCA provides efficient dimensionality reduction")
66 print("    with minimal accuracy loss")
67 print("=" * 80)
```

FINAL COMPREHENSIVE RESULTS SUMMARY

📊 TASK 1: PCA ON IRIS DATASET

- ✅ Applied PCA to reduce IRIS dataset from 4 to 3 dimensions
- ✅ Captured 99.48% of total variance
- ✅ Created 3D visualization showing clear class separation

📊 TASK 2: EIGENFACES ON LFW DATASET

Dataset: 158 people, 4324 total images (64x64 pixels)
Train/Test Split: 3459 / 865 images
Original Features: 4096 pixels per image

DIMENSIONALITY REDUCTION RESULTS

Variance	Components	Compression	Features Kept
No PCA	4096	0%	100%
95%	178	95.7%	4.3%
97%	267	93.5%	6.5%

CLASSIFICATION ACCURACY RESULTS

Classifier	No PCA	PCA 95%	PCA 97%
SVM	25.66%	26.47%	26.47%
Random Forest	22.66%	17.69%	16.88%
KNN	18.61%	19.19%	18.84%
Logistic Regression	45.43%	44.62%	45.20%

🏆 BEST PERFORMING CLASSIFIER ANALYSIS

- ✅ Best without PCA: Logistic Regression (45.43%)
 - ✅ Best with 95% variance: Logistic Regression (44.62%)
 - ✅ Best with 97% variance: Logistic Regression (45.20%)
- 🏆 OVERALL BEST PERFORMER: Logistic Regression (No PCA)
Accuracy: 45.43%

📋 KEY FINDINGS & CONCLUSIONS

- Logistic Regression consistently outperforms all other classifiers
- PCA achieves 93-95% compression while maintaining similar accuracy
- The first eigenface captures 27.55% of total variance
- 270 components (97% variance) vs 180 components (95% variance)
- For this dataset, PCA provides efficient dimensionality reduction with minimal accuracy loss

```
1 # Create results DataFrame
2 results_data = {
```

```

3     'Classifier': [],
4     'No_PCA_Accuracy': [],
5     'PCA_95_Accuracy': [],
6     'PCA_97_Accuracy': [],
7     'No_PCA_Features': [],
8     'PCA_95_Features': [],
9     'PCA_97_Features': []
10 }
11
12 for clf_name in classifiers.keys():
13     results_data['Classifier'].append(clf_name)
14     results_data['No_PCA_Accuracy'].append(f"{baseline_results[clf_name]*100:.2f}%")
15     results_data['PCA_95_Accuracy'].append(f"{results_95[clf_name]*100:.2f}%")
16     results_data['PCA_97_Accuracy'].append(f"{results_97[clf_name]*100:.2f}%")
17     results_data['No_PCA_Features'].append(4096)
18     results_data['PCA_95_Features'].append(180)
19     results_data['PCA_97_Features'].append(270)
20
21 results_df = pd.DataFrame(results_data)
22 results_df.to_csv('classification_results.csv', index=False)
23
24 print("✅ Results saved to 'classification_results.csv'")
25 print("\nResults Preview:")
26 print(results_df.to_string(index=False))
27
28 # Create summary statistics
29 print("\n" + "=" * 80)
30 print("📊 SUMMARY STATISTICS")
31 print("=" * 80)
32
33 summary_data = {
34     'Metric': [
35         'Total Images',
36         'Training Images',
37         'Test Images',
38         'Number of People',
39         'Image Dimensions',
40         'Original Features',
41         'PCA 95% Components',
42         'PCA 97% Components',
43         'PCA 95% Compression',
44         'PCA 97% Compression',
45         'Best Classifier',
46         'Best Accuracy (No PCA)',
47         'Best Accuracy (95% PCA)',
48         'Best Accuracy (97% PCA)'
49     ],
50     'Value': [
51         f"{X.shape[0]}",
52         f"{X_train.shape[0]}",
53         f"{X_test.shape[0]}",
54         f"{len(selected_people)}",
55         f"{img_height}x{img_width}",
56         f"{X_train.shape[1]}",
57         f"{X_train_pca_95.shape[1]}",
58         f"{X_train_pca_97.shape[1]}",
59         f"{(1-X_train_pca_95.shape[1]/X_train.shape[1])*100:.1f}%",
60         f"{(1-X_train_pca_97.shape[1]/X_train.shape[1])*100:.1f}%",
61         "Logistic Regression",
62         f"{baseline_results['Logistic Regression']*100:.2f}%",
63         f"{results_95['Logistic Regression']*100:.2f}%",
64         f"{results_97['Logistic Regression']*100:.2f}%"
65     ]
66 }
67
68 summary_df = pd.DataFrame(summary_data)
69 summary_df.to_csv('assignment_summary.csv', index=False)
70
71 print("✅ Summary saved to 'assignment_summary.csv'")
72 print("\nSummary Preview:")
73 print(summary_df.to_string(index=False))
74 print("=" * 80)
75

```

✅ Results saved to 'classification_results.csv'

Results Preview:

Classifier	No PCA Accuracy	PCA 95 Accuracy	PCA 97 Accuracy	No PCA Features	PCA 95 Features	PCA 97 Features
------------	-----------------	-----------------	-----------------	-----------------	-----------------	-----------------